

Multilingual Speech Pipeline: Deployment Notes

Internal engineering reference covering ingestion, model evaluation, infrastructure, and deployment considerations for low-resource ASR and TTS pipelines across Urdu, Pashto, and Punjabi. This document is maintained by the speech platform team and updated after each quarterly model refresh.

1. Overview

This document summarizes the current state of the multilingual speech pipeline, including data collection practices, model architecture choices, infrastructure configuration, and known failure modes across supported languages. It is intended as a living reference for engineers onboarding onto the speech team, and as a record of decisions made during the current model generation.

The pipeline processes raw audio through a shared front-end feature extractor before branching into language-specific acoustic models. Text normalization and tokenization are handled separately for each language due to differing orthographic conventions, script variants, and dialectal spelling inconsistencies across data sources.

Three languages are currently in production: Urdu, Pashto, and Punjabi. A fourth language, Sindhi, is in early data collection and is not covered in this document.

2. Data Ingestion

2.1 Audio Collection

Audio data is sourced from three channels: crowd-sourced recordings, licensed broadcast archives, and synthetic augmentation. Each source is tagged with provenance metadata at ingestion time so downstream evaluation can be segmented by data origin, which has proven useful for tracing quality regressions back to a specific batch.

Crowd-sourced recordings undergo automatic quality filtering based on signal-to-noise ratio and clipping detection before being queued for manual review. Roughly 30% of submitted crowd-sourced audio is rejected at this stage, most commonly due to background noise exceeding the configured threshold.

Licensed broadcast archives provide higher average audio quality but skew toward formal, scripted speech, which limits their usefulness for training conversational robustness. The team is actively working to balance the training mix toward a higher proportion of informal speech.

2.2 Text Corpus Preparation

Text corpora are normalized using language-specific rules. Urdu and Pashto both require handling of the Perso-Arabic script's contextual letter shaping, while Punjabi ingestion must support both Gurmukhi and Shahmukhi scripts depending on source region.

- Strip non-standard diacritics inconsistently applied by different data sources
- Normalize digit representations (Eastern Arabic-Indic vs. Western Arabic numerals)
- Segment sentences using script-aware boundary detection rather than Latin punctuation heuristics
- Flag and quarantine mixed-script lines for manual review before inclusion in training data

2.3 Metadata Schema

Every ingested utterance is stored with a consistent metadata record so that downstream training and evaluation code can filter and stratify samples without re-parsing raw files. The following fields are considered required, not optional, for any sample entering the training pipeline:

1. source_channel — crowd-sourced, broadcast, or synthetic
2. language_code — ISO 639-3 plus an internal dialect tag where applicable
3. script_variant — for Punjabi specifically, Gurmukhi or Shahmukhi
4. qc_status — passed, flagged, or rejected, with a reason code
5. recording_date and ingestion_date — kept separate since batches are sometimes ingested well after recording

3. Model Architecture

3.1 Shared Front End

All three languages share a single front-end feature extractor, trained jointly on the combined multilingual corpus. This was a deliberate choice to allow low-resource languages like Pashto to benefit from acoustic patterns learned primarily from the larger Urdu dataset, under the hypothesis that low-level acoustic features transfer reasonably well across related languages even when higher-level phonology differs.

Empirically, this transfer has held up reasonably well for Pashto but less so for Punjabi, where tonal distinctions not present in Urdu or Pashto appear to be partially lost in the shared representation. This is an open area of investigation for the next model generation.

3.2 Language-Specific Branches

After the shared front end, each language routes to its own acoustic model, fine-tuned independently. This branching point was chosen empirically by comparing WER across several candidate architectures where the split occurred earlier or later in the network; splitting immediately after the front end gave the best balance between shared representation benefit and language-specific specialization.

4. Model Evaluation

Evaluation is conducted per-language using held-out test sets that are refreshed quarterly to avoid overfitting to a static benchmark. Automatic Speech Recognition (ASR) is scored using Word Error Rate (WER), while Text-to-Speech (TTS) output is scored using Mean Opinion Score (MOS) from human raters on a 1-5 scale.

Table 4.1: ASR and TTS Evaluation Metrics by Language

Language	ASR WER (%)	TTS MOS	Training Hours
Urdu	14.2	4.1	320
Pashto	21.7	3.6	140
Punjabi	18.3	3.9	210

Pashto currently shows the highest WER of the three supported languages, which the team attributes to limited training hours relative to Urdu and greater dialectal variation across regions. Punjabi sits between the two, likely reflecting its larger but still comparatively modest training set.

4.1 Error Breakdown

Beyond aggregate WER, the team tracks a breakdown of error types to understand where model improvements should be targeted. Substitution errors dominate for both Urdu and Pashto, but the gap between the two languages is largest in this category, consistent with Pashto's greater dialectal pronunciation variance.

Table 4.2: Error Type Breakdown for Urdu and Pashto

Error Type	Urdu %	Pashto %	Primary Cause
------------	--------	----------	---------------

Substitution	8.1	12.4	Dialectal pronunciation variance
Deletion	3.2	5.8	Fast speech / phoneme dropping
Insertion	2.9	3.5	Background noise misattribution

4.2 Human Evaluation Protocol

MOS scores are collected from a pool of at least twelve native-speaker raters per language, with each sample rated by a minimum of three raters and outlier scores (more than two standard deviations from the sample mean) discarded before averaging. Raters are shown the synthesized audio alongside the source text and asked to rate naturalness on a 1-5 scale without being told which system produced the sample.

5. Infrastructure

5.1 Serving Configuration

The production pipeline is deployed as a set of independently scalable services, allowing the team to allocate GPU capacity toward whichever language is experiencing higher load without over-provisioning the others. The current reference configuration is summarized below.

Table 5.1: Reference Serving Configuration

Component	Instance Type	Avg Latency
Feature extractor	c6i.2xlarge	40ms
Urdu acoustic model	g5.xlarge (GPU)	310ms
Pashto acoustic model	g5.xlarge (GPU)	295ms
Punjabi acoustic model	g5.xlarge (GPU)	330ms
TTS vocoder	g5.xlarge (GPU)	480ms
Post-processing / normalization	c6i.large	15ms

5.2 Latency Targets

Production deployment targets end-to-end latency under 800ms for streaming ASR and under 1.2s for TTS synthesis of a single sentence. These targets are measured on the reference inference server configuration documented above, under a simulated load of 50 concurrent requests per language.

5.3 Fallback Behavior

When confidence scores fall below the configured threshold, the pipeline falls back to a smaller, faster model rather than failing the request outright. This trades some accuracy for reliability under load, and the threshold is tuned separately per language based on observed confidence calibration, since the three acoustic models are not equally well calibrated against their own confidence outputs.

6. Known Limitations

6.1 Code-Switching

Code-switching between Urdu and English within a single utterance remains a significant source of errors, particularly in informal speech. The current tokenizer does not reliably detect script boundaries mid-utterance, which can cause both ASR misrecognition and unnatural TTS prosody at the switch point.

Further work is planned to address this through a dedicated code-switch detection module ahead of the language-specific branching stage. A prototype using audio-level prosodic cues, rather than text-level script detection, is currently under evaluation.

6.2 Script Ambiguity in Punjabi

Punjabi source material arrives in both Gurmukhi and Shahmukhi script depending on regional origin, and a nontrivial fraction of crowd-sourced text data mixes the two within a single document, which complicates both training data preparation and evaluation set construction.

6.3 Dialect Coverage Gaps

Current training data for Pashto skews toward a small number of regional dialects overrepresented in available broadcast archives. Speakers from underrepresented dialect regions report noticeably higher error rates in informal user feedback, though this has not yet been quantified through a structured evaluation.

7. Training Pipeline

7.1 Data Splitting Strategy

Training, validation, and test splits are constructed at the speaker level rather than the utterance level, meaning no speaker appears in more than one split. This was adopted after an earlier utterance-level split was found to substantially overestimate held-out performance, since the model could partially memorize speaker-specific acoustic characteristics rather than generalizing across speakers.

Speaker-level splitting is straightforward for crowd-sourced and broadcast data, where speaker identity is known, but is not always possible for synthetic augmentation samples, which are excluded from the test split entirely as a precaution.

7.2 Curriculum and Fine-Tuning Order

The shared front end is pretrained first on the full multilingual corpus before any language-specific fine-tuning begins. Language branches are then fine-tuned in a fixed order: Urdu first, given its larger dataset and role as the primary transfer source, followed by Punjabi, then Pashto.

This ordering was chosen empirically; fine-tuning Pashto before Urdu was tested and found to degrade Urdu performance more than the reverse ordering degraded Pashto, likely because Urdu's larger dataset makes it more resistant to catastrophic forgetting during sequential fine-tuning.

7.3 Augmentation

Synthetic augmentation is used primarily to address dialect and noise-condition gaps in the natural data rather than to increase raw volume. Current augmentation techniques in use include:

- Speed and pitch perturbation within a range validated to preserve intelligibility
- Background noise injection sampled from a library of real-world recording environments
- Text-to-speech round-tripping for underrepresented dialect regions, used cautiously given the risk of reinforcing the TTS model's own biases back into the ASR training set

8. Monitoring and Alerting

8.1 Production Metrics

Each language pipeline reports latency, error rate proxies (via confidence score distributions, since ground-truth WER is not available on live traffic), and request volume to the shared monitoring dashboard. Alerts are configured per language rather than globally, since the three languages have different baseline confidence distributions and a shared threshold would either be too sensitive for one language or too lax for another.

8.2 Drift Detection

A weekly job compares the confidence score distribution of live traffic against a reference distribution captured at the time of the last model deployment. A significant leftward shift, indicating the model is less confident on average, triggers a review, since this has historically preceded user-reported quality regressions by one to two weeks in past incidents.

This drift detection approach is intentionally conservative and produces some false positives, which the team considers an acceptable tradeoff given the cost of an undetected quality regression reaching a large fraction of production traffic before being caught.

9. Incident History

This section summarizes notable production incidents for institutional memory. Full postmortems are linked in the internal incident tracker; this document retains only a brief summary of cause and resolution.

9.1 Punjabi Script Misdetection (Q1)

A batch of Shahmukhi-script Punjabi data was incorrectly tagged as Gurmukhi during an automated ingestion run following a change to the source data provider's file naming convention, which the script-detection heuristic relied on. This caused a measurable quality regression in Punjabi TTS output for roughly one week before being caught by the drift detection system described in Section 8.2.

The resolution included both a fix to the ingestion tagging logic and a move away from relying on file naming conventions as a script-detection signal, in favor of content-based script detection run at ingestion time regardless of source metadata.

9.2 Latency Regression from Shared Front-End Update (Q2)

An update to the shared front-end feature extractor, intended to improve the Punjabi tonal transfer issue described in Section 3.1, incidentally increased front-end inference time by roughly 35%, pushing Urdu end-to-end latency above the 800ms target under peak load.

This was resolved by moving the updated front end to a larger instance type rather than reverting the model change, since the underlying accuracy improvement was considered worth the added infrastructure cost.

10. Roadmap

The following items are planned for the next two quarters, in approximate priority order:

1. Ship the audio-level code-switch detection prototype to a shadow deployment for offline evaluation against production traffic
2. Expand Pashto dialect coverage through a targeted crowd-sourcing campaign in underrepresented regions
3. Begin early data collection for Sindhi to support a future fourth production language
4. Re-evaluate the shared front-end architecture given the tonal transfer gap observed for Punjabi
5. Move content-based script detection upstream of all ingestion paths, closing the gap identified in the Q1 incident
6. Investigate whether speaker-level splitting should be extended to a stricter household-level or region-level split for crowd-sourced data

Progress against this roadmap is reviewed at the end of each quarter alongside the refreshed evaluation benchmarks described in Section 4.

11. Data Privacy and Compliance

11.1 Consent and Retention

Crowd-sourced audio contributors provide explicit consent at collection time covering both model training use and internal quality review by human raters. Consent records are stored separately from the audio data itself and are

checked programmatically before any batch is admitted into the training pipeline, rather than being assumed valid based on the source channel alone.

Raw audio is retained for a maximum of eighteen months following collection, after which it is deleted while derived features and model weights, which cannot be reversed back into intelligible audio, are retained indefinitely. This retention policy was set in coordination with the legal team and differs slightly from the licensed broadcast archive data, which follows retention terms specified in the individual licensing agreements rather than the internal default.

11.2 Regional Data Handling

Punjabi data collected from sources indicating Pakistani origin and data indicating Indian origin are currently stored in the same pipeline but tagged separately, in anticipation of possible future regulatory requirements around cross-border data handling. No technical separation is currently enforced beyond this tagging, but the team has flagged this as a likely area of future work given increasing regulatory attention to speech data specifically, as opposed to text data, in the region.

11.3 Rater Access Controls

Human raters involved in MOS evaluation and manual quality review access audio samples through a restricted internal tool rather than raw file access, and are contractually prohibited from retaining or exporting any sample they review. Rater accounts are audited quarterly against the current active rater roster to ensure access is revoked promptly when a rater's engagement ends.

12. Team Ownership and Escalation

12.1 Component Ownership

Ingestion and data pipeline components are owned by the data engineering subteam, while acoustic model architecture and training are owned by the modeling subteam. The shared front end sits at the boundary between the two and changes to it require sign-off from both subteams, given the Q2 latency incident described in Section 9.2 where a modeling-driven change had unanticipated infrastructure consequences.

Infrastructure and serving configuration, including the instance types and scaling policy referenced in Section 5.1, are owned by the platform reliability subteam, who are the primary responders for the latency and drift alerts described in Section 8.

12.2 Escalation Path

Production incidents affecting live traffic should be escalated through the standard on-call rotation rather than directly to individual component owners, since the on-call engineer has the context to determine which subteam's involvement is needed and can page accordingly. Data quality issues discovered outside of an active incident, such as a mistagged batch found during routine review, should instead be filed through the data quality ticket queue rather than paged, unless the issue is already affecting production traffic.

This distinction has occasionally been a source of confusion for newer team members, since a data quality issue can escalate into a production incident if left unaddressed, as happened in the Q1 script misdetection case. When in doubt, engineers are encouraged to file both a ticket and a page rather than deliberate too long over which path is more appropriate.